
Intrinsic Token Weighting for Credit Assignment in Language Model Reinforcement Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Policy gradient methods for LLM reinforcement learning often rely on value-
2 function critics, but language-generation prefixes are non-Markov and poorly
3 suited for stable value estimation. Recent critic-free approaches such as GRPO
4 and RLOO avoid value networks, yet they still broadcast the same trajectory-level
5 signal uniformly across tokens. We study an alternative: intrinsic token weighting,
6 which redistributes a prompt-level advantage across tokens using salience signals
7 already available from the policy, including token surprisal and local entropy
8 reduction. These methods require no additional networks, no tree construction,
9 and no reward models. To make the experimental setup concrete and reproducible,
10 the repository implements Qwen-based experiments on GSM8K and MBPP with
11 exact-answer and unit-test rewards. The resulting codebase provides a direct way
12 to evaluate whether redistributing credit via intrinsic distributional signals is a
13 practical alternative to uniform token updates in critic-free language RL.

14 1 Introduction

15 Reinforcement learning has become an important component of large language model (LLM) training
16 pipelines, particularly for alignment, preference optimization, and reasoning tasks. Most contem-
17 porary approaches rely on policy gradient methods augmented with advantage estimation, typically
18 implemented via learned value functions or generalized advantage estimation. These techniques orig-
19 inate from classical reinforcement learning settings in which states are well-defined, approximately
20 Markov, and admit stable value estimates.

21 Applying this framework to language generation introduces a fundamental mismatch. In autore-
22 gressive LLMs, the “state” is a partial sequence of tokens. Such prefixes are not Markov, do not
23 summarize progress toward task completion, and can vary widely in semantic meaning despite
24 superficial similarity. Consequently, estimating a value function over prefixes is both statistically
25 challenging and conceptually ill-defined. Empirically, value heads in language RL are often brittle,
26 sensitive to distributional shift, and prone to introducing bias that outweighs their variance-reduction
27 benefits.

28 Motivated by these issues, recent work has increasingly moved away from learned critics in LLM
29 reinforcement learning. Critic-free, outcome-based methods such as RLOO and GRPO replace value
30 estimation with batch-level baselines, achieving stable training on verifiable reasoning tasks such as
31 math and code. However, these approaches still implicitly assume uniform credit assignment across
32 tokens: every action in a trajectory receives the same reward signal, modulated only by a scalar
33 baseline.

34 This raises a natural question: *if we no longer estimate values over prefixes, how should trajectory-*
35 *level reward be distributed across tokens?* Uniform credit assignment is simple, but it ignores

the fact that different tokens play very different roles in generation. Some tokens correspond to high-uncertainty decisions, strategy commitments, or branching points in reasoning, while others are low-entropy continuations or stylistic filler. Treating all tokens equally risks diluting the learning signal and obscuring the mechanisms by which policies improve.

In this work, we study token-level credit redistribution as a mechanism for on-policy LLM reinforcement learning without value functions. Rather than estimating advantages or prefix values, we propose to weight token-level policy gradients using intrinsic signals derived directly from the model’s predictive distribution. These include token surprisal and changes in predictive entropy, which naturally reflect decision difficulty and commitment during generation. Importantly, these signals are inexpensive to compute, require no additional networks or tree structures, and make no assumptions about the existence of a meaningful value function over prefixes.

To make this question concrete, the repository now implements a reproducible experimental stack around Qwen models on GSM8K and MBPP. The code compares uniform token updates against RLOO/GRPO baselines with surprisal-based and entropy-reduction-based intrinsic weighting, all under matched prompt formatting and optimization settings. This turns the paper’s experimental section into a directly runnable benchmark rather than a placeholder description.

Overall, this paper reframes advantage estimation as one instantiation of a broader credit assignment problem in language reinforcement learning. In settings where state semantics are weak and rewards are trajectory-level, redistributing credit using intrinsic distributional signals is a plausible alternative to value-based methods; the implemented experiments are designed to test that claim directly.

2 Related Work

2.1 From RLHF to RLVR

Modern language-model post-training inherits its optimization machinery from policy-gradient reinforcement learning, from REINFORCE [25] to trust-region and clipped-surrogate methods such as TRPO and PPO [16, 18]. Advantage estimation, especially GAE, became the standard variance-reduction device in continuous-control RL by learning value functions over states or prefixes [17]. In language modeling, these ideas entered mainstream use through RLHF systems that optimize sequence-level rewards derived from human preferences or reward models [5, 21, 14].

The recent reasoning-model wave shifted part of the field from preference-based RLHF toward reinforcement learning with verifiable rewards (RLVR), where supervision is often a deterministic correctness signal on math or code tasks. DeepSeekMath introduced GRPO as a practical critic-free alternative for these settings [19], and DeepSeek-R1 popularized large-scale RL-only or RL-dominant reasoning pipelines [7]. Subsequent surveys document how quickly this regime became the default template for open reasoning-model replication and extension [29]. This transition matters for our setting because RLVR makes sparse outcome rewards and long reasoning trajectories central, thereby exposing the credit-assignment problem more directly than earlier short-form RLHF tasks.

2.2 Critic-Based and Critic-Free Policy Optimization

The classical solution to delayed reward is to learn a critic or value function and convert returns into token- or prefix-level advantages. PPO-style RLHF pipelines follow this recipe, but the value-estimation problem is particularly awkward for text generation because prefixes are high-dimensional, non-Markov, and semantically heterogeneous. A growing body of work therefore revisits whether value heads are necessary at all in LLM post-training.

ReMax replaces learned critics with a greedy baseline and shows that much of PPO’s practical benefit can be retained with a simpler REINFORCE-style objective [12]. Back to Basics systematically compares PPO with RLOO-style estimators and finds that critic-free methods can match or exceed value-based baselines in RLHF [1]. REINFORCE++ further strengthens this line by improving robustness to prompt and reward-model variation without reintroducing a learned value function [9]. In reasoning-focused RLVR, GRPO normalizes rewards across a group of sampled responses rather than learning prefix values [19], and later work refines this family with theoretical analyses, off-policy variants, and sequence-level formulations such as GSPO [13, 30]. These methods substantially

86 simplify the optimization stack, but they generally still broadcast a trajectory-level signal across all
87 tokens once a scalar baseline is fixed.

88 2.3 Reasoning-Specific Analyses of RLVR

89 Another recent line asks why critic-free RLVR works so well for reasoning in the first place. A
90 Minimalist Approach to LLM Reasoning argues that strong reasoning gains can emerge from a
91 comparatively simple recipe that moves from rejection sampling to REINFORCE-style online updates,
92 suggesting that some of the field’s complexity is optional rather than essential [26]. Complementarily,
93 Wen et al. analyze RLVR theoretically and empirically, arguing that answer-level verifiable rewards
94 can nonetheless incentivize correct intermediate reasoning early in a trajectory [24]. These results
95 are important because they show that uniform outcome-level objectives already contain nontrivial
96 reasoning signal.

97 At the same time, they do not eliminate the question of *which* tokens should absorb that signal. Recent
98 empirical analysis of RLVR training dynamics suggests that improvements are disproportionately
99 driven by a minority of high-entropy “forking” tokens, and that restricting updates to this subset
100 can remain competitive or even outperform full-token updates in some settings [23]. This is the
101 closest recent line to our work conceptually: it supports the claim that token salience is highly
102 non-uniform during reasoning RL. Our work differs in scope and mechanism, however, by studying
103 general intrinsic weighting schemes based on token surprisal and entropy change rather than a hard
104 top-entropy masking rule or an empirical analysis specific to one RLVR recipe.

105 2.4 Fine-Grained Credit Assignment Beyond Uniform Token Updates

106 A large parallel literature attempts to make supervision denser than a single end-of-trajectory reward.
107 One family learns explicit token- or process-level reward estimators. Preference-grounded Token-level
108 Guidance derives token-level signals from preference data for fine-tuning [27], while Discriminative
109 Policy Optimization learns token-level Q-style reward models from preferences without requiring
110 fine-grained annotation [4]. PRIME pushes this direction further for reasoning tasks by constructing
111 implicit process rewards and updating process reward models online from rollouts and outcome
112 labels [6]. These methods can produce rich token-level feedback, but they rely on an auxiliary
113 reward-modeling component that our approach intentionally avoids.

114 Another family redistributes outcome rewards algorithmically rather than by training a dense reward
115 model. RED derives token-level rewards by redistributing holistic reward-model scores over a
116 trajectory [10]. SCAR uses Shapley-inspired attribution to estimate token or span contributions from
117 sequence-level feedback [2, 20]. TEMPO uses groups of sampled solutions to build a prefix tree and
118 compute nonparametric prefix values, yielding branch-sensitive corrections without a learned critic
119 [22]. GRPO- λ introduces eligibility traces and lambda-returns into critic-free RLVR to propagate
120 outcome information backward along the sequence [15]. Compared with these approaches, our focus
121 is narrower and cheaper: we do not train auxiliary reward models, estimate Shapley values, construct
122 trees, or generate corrective interventions, but instead ask how far one can get by reweighting the
123 existing policy-gradient terms using intrinsic distributional signals already present in the forward
124 pass.

125 2.5 Alternative Action Granularities and Token Selection

126 Several papers attack the same underlying problem by changing the optimization unit rather than
127 by redesigning the reward. MA-RLHF introduces macro actions, grouping tokens into higher-level
128 units to shorten the temporal distance between decisions and rewards [3]. GSPO moves importance
129 weighting and clipping from the token level to the sequence level for greater stability in large-scale
130 RL training [30]. VinePPO replaces learned value networks with Monte Carlo vine-style estimates
131 over continuations, seeking better credit propagation through structured rollout reuse [11]. These
132 methods reinforce the broader point that the granularity of optimization matters as much as the reward
133 definition.

134 Outside on-policy RL proper, token-selective optimization has also been explored in adjacent
135 paradigms. ConfPO emphasizes preference-critical tokens based on policy confidence in preference
136 optimization [28], and token weighting for long-range language modeling shows that non-uniform

token emphasis can improve optimization even in supervised settings [8]. Together with high-entropy token analyses in RLVR [23], these results strongly suggest that uniform token treatment is an arbitrary design choice rather than a necessity.

2.6 Positioning of This Work

The literature now makes three points clear. First, critics are not strictly required for effective LLM post-training: critic-free estimators such as RLOO, ReMax, GRPO, and GSPO are already competitive in both RLHF and RLVR [1, 12, 19, 30]. Second, many researchers have concluded that trajectory-level rewards are too coarse and have introduced denser supervision through process reward models, redistribution rules, tree structures, or temporal traces [10, 6, 22, 2, 15]. Third, recent reasoning-focused analyses indicate that only a subset of tokens may be responsible for most of the useful RL signal [24, 23].

Our paper sits at the intersection of these trends but occupies a distinct point in the design space. We study *intrinsic* token weighting for on-policy language RL: credit is redistributed at the token level using signals derived from the policy’s own predictive distribution, without learned value heads, external reward models, process annotations, tree construction, Shapley-style counterfactual attribution, or corrective rollouts. This positioning is important for reviewer clarity: our contribution is not another dense-reward model or another GRPO variant, but a minimal credit-redistribution mechanism that can be layered onto critic-free policy gradients while preserving their computational simplicity.

3 Methods

We consider on-policy reinforcement learning for autoregressive language models with trajectory-level rewards. Let π_θ denote a language model parameterized by θ , generating a completion $y = (y_1, \dots, y_T)$ conditioned on a prompt x . After sampling a full trajectory, the model receives a scalar reward $R(x, y) \in \mathbb{R}$ computed by an external verifier, such as exact-answer correctness or unit-test pass rate. Our objective is

$$J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)}[R(x, y)], \quad (1)$$

where $\mathcal{D}$ denotes the prompt distribution. We focus on the regime most relevant to recent reasoning-model training: sparse outcome rewards, on-policy sampling, and no learned value function.

3.1 Policy Gradient with Trajectory-Level Reward

For a fixed prompt x , the standard REINFORCE estimator is

$$g_{\text{rf}}(x, y) = R(x, y) \sum_{t=1}^T \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}, x). \quad (2)$$

In practice, a baseline $b(x)$ is subtracted to reduce variance without changing the expected gradient as long as $b(x)$ does not depend on the sampled action at token t :

$$g_{\text{base}}(x, y) = (R(x, y) - b(x)) \sum_{t=1}^T \nabla_\theta \log \pi_\theta(y_t \mid y_{<t}, x). \quad (3)$$

We use prompt-level multi-sample baselines. Given K sampled completions $\{y^{(i)}\}_{i=1}^K$ for the same prompt x with rewards $\{R^{(i)}\}_{i=1}^K$:

$$b_{\text{RLOO}}^{(i)}(x) = \frac{1}{K-1} \sum_{j \neq i} R^{(j)} \quad (4)$$

is the leave-one-out baseline used in RLOO-style estimators, while GRPO-style normalization can be written as

$$A_{\text{GRPO}}^{(i)}(x) = \frac{R^{(i)} - \mu_R(x)}{\sigma_R(x) + \varepsilon}, \quad (5)$$

where $\mu_R(x)$ and $\sigma_R(x)$ are the within-prompt mean and standard deviation of the K rewards. In both cases, the resulting scalar advantage is then broadcast uniformly over all tokens in the sampled trajectory. This scalar-broadcast assumption is precisely what we relax.

3.2 Token-Level Credit Redistribution

We introduce token-level weights $w_t(x, y)$ that redistribute a trajectory-level advantage across tokens:

$$g_w(x, y) = A(x, y) \sum_{t=1}^T w_t(x, y) \nabla_{\theta} \log \pi_{\theta}(y_t \mid y_{<t}, x), \quad (6)$$

where $A(x, y)$ denotes either $R(x, y) - b_{\text{RLOO}}(x)$ or $A_{\text{GRPO}}(x, y)$ depending on the baseline choice.

The weights satisfy

$$\sum_{t=1}^T w_t(x, y) = 1, \quad w_t(x, y) \geq 0. \quad (7)$$

This normalization keeps the total update magnitude comparable across weighting schemes and isolates the effect of credit redistribution from simple rescaling.

Uniform token credit corresponds to $w_t = 1/T$ for all t . Our goal is to replace this uniform allocation with *intrinsic* weights derived from quantities already produced by the policy during generation. We do not train token-level reward models, estimate prefix values, or build explicit search trees.

3.3 Intrinsic Token-Weighting Mechanisms

We define each method through an unnormalized salience score $\alpha_t(x, y) \geq 0$ and then normalize within each trajectory:

$$w_t(x, y) = \frac{\alpha_t(x, y) + \varepsilon}{\sum_{k=1}^T (\alpha_k(x, y) + \varepsilon)}, \quad (8)$$

with a small $\varepsilon > 0$ to avoid degenerate all-zero cases. In implementation, the weights are treated as *detached* scalars when multiplying the policy-gradient term, so the update does not introduce second-order derivatives through the weighting function itself. This keeps the optimization rule close in spirit and cost to standard critic-free policy gradients.

Uniform Weighting (Baseline).

$$w_t^{\text{uniform}} = \frac{1}{T}. \quad (9)$$

Combined with an RLOO or GRPO baseline, this recovers the standard critic-free estimator that assigns identical credit to every token in the sampled completion.

Surprisal Weighting. Our first intrinsic score is token surprisal,

$$\alpha_t^{\text{sup}}(x, y) = -\log \pi_{\theta}(y_t \mid y_{<t}, x). \quad (10)$$

This emphasizes low-probability decisions under the current policy. Intuitively, these are tokens where the model commits to a less routine continuation and where uniform credit assignment may be most diluted by predictable filler tokens.

Entropy-Reduction Weighting. Our second intrinsic score measures local entropy reduction. Let

$$H_t^{\text{pre}}(x, y) = -\sum_v \pi_{\theta}(v \mid y_{<t}, x) \log \pi_{\theta}(v \mid y_{<t}, x) \quad (11)$$

denote the predictive entropy before sampling token y_t , and let

$$\Delta H_t^{\text{ent}}(x, y) = \max(0, H_t^{\text{pre}}(x, y) - H_{t+1}^{\text{pre}}(x, y)) \quad (12)$$

for $t < T$. For the final token we set $\Delta H_T^{\text{ent}} = 0$. We then use

$$\alpha_t^{\text{ent}}(x, y) = \Delta H_t^{\text{ent}}(x, y). \quad (13)$$

This score highlights tokens after which the model’s next-step distribution becomes substantially more concentrated. Such events often correspond to commitment points in reasoning, where one branch of continuation becomes much more likely than its alternatives.

Length-Robust Normalization. Because reasoning trajectories can vary substantially in length, all weights are normalized within each sampled completion rather than across the minibatch. This ensures that longer trajectories do not automatically receive larger aggregate updates simply because they contain more token positions with nonzero salience. The comparison between weighting schemes therefore reflects *where* credit is assigned within a trajectory, not how much total credit a longer sequence receives.

Batch-Level Baselines. All weighting schemes can be paired with either RLOO or GRPO-style prompt-level baselines. Unless otherwise specified, we use the same rollout groups, optimizer, sampling hyperparameters, and baseline family across methods. This isolates token redistribution as the only algorithmic difference.

3.4 Relationship to Existing Methods

Our formulation recovers standard critic-free RL as a special case: uniform weighting plus a prompt-level batch baseline yields the usual RLOO/GRPO-style estimator. It also clarifies how our method differs from nearby alternatives. Unlike critic-based PPO, we do not learn a value head. Unlike RED, PRIME, or token-level reward-model methods, we do not construct dense rewards from an auxiliary model. Unlike TEMPO or related tree-based methods, we do not build prefix graphs or estimate branch values. Unlike hard token-selection methods based on top-entropy masking, we retain a dense token update but modulate it continuously using intrinsic salience scores.

The resulting estimator is intentionally minimal. It changes only the within-trajectory allocation of a scalar on-policy advantage, using quantities already available from the forward pass. This makes it easy to layer on top of existing critic-free RLVR pipelines while preserving their computational simplicity.

4 Theoretical Interpretation

We provide an interpretation of intrinsic token weighting as a variance-control and credit-redistribution mechanism for policy gradient estimation in language models. Our goal is not to derive new convergence guarantees, but to clarify how token weighting relates to advantage estimation and why it can be effective without learning value functions.

4.1 Token Weighting as Credit Redistribution

Consider the standard REINFORCE estimator with a batch-level baseline:

$$g = (R - b) \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(y_t \mid y_{<t}, x). \quad (14)$$

This estimator assigns equal credit to all tokens in a trajectory. Introducing token weights yields:

$$g_w = (R - b) \sum_{t=1}^T w_t(y) \nabla_{\theta} \log \pi_{\theta}(y_t \mid y_{<t}, x), \quad (15)$$

where $\sum_t w_t = 1$.

Importantly, this transformation does not change the reward signal itself; it redistributes how the trajectory-level signal is attributed to individual decisions. When w_t depends only on quantities available at sampling time (e.g., log-probabilities or entropy), the estimator remains on-policy and does not require learning additional functions.

4.2 Relationship to Advantage Estimation

Advantage-based methods can be interpreted as implicitly defining token-level weights via a learned value function:

$$A_t = R - V(y_{<t}, x). \quad (16)$$

241 In this view, the contribution of each token is scaled according to how much the observed outcome
 242 deviates from an estimated expectation conditioned on the prefix.

243 However, this interpretation relies on the existence of a meaningful value function over prefixes. In
 244 language generation, prefixes are non-Markov and do not form a stable state space, making $V(y_{<t}, x)$
 245 difficult to estimate and prone to bias. Token weighting offers an alternative: rather than estimating
 246 expected returns from prefixes, it emphasizes tokens based on intrinsic measures of decision salience,
 247 such as uncertainty or information gain.

248 4.3 Connection to Control Variates

249 From a statistical perspective, subtracting a baseline b is a form of control variate that reduces variance
 250 without affecting unbiasedness. Token weighting can be viewed as a complementary mechanism that
 251 reshapes the contribution of individual score-function terms:

$$\nabla_{\theta} \log \pi_{\theta}(y) = \sum_{t=1}^T \nabla_{\theta} \log \pi_{\theta}(y_t \mid y_{<t}, x). \quad (17)$$

252 Uniform weighting treats all score-function terms equally, regardless of their variance or relevance.
 253 Intrinsic weighting schemes effectively reweight these terms, down-weighting low-variance or low-
 254 impact contributions (e.g., high-probability continuation tokens) and emphasizing high-variance or
 255 high-impact decisions (e.g., low-probability or uncertainty-reducing tokens). While this reweighting
 256 introduces bias relative to the uniform estimator, it can substantially reduce variance, leading to
 257 improved optimization dynamics in practice.

258 4.4 Interpretation of Specific Weighting Schemes

259 **Surprisal Weighting.** Surprisal-based weights emphasize tokens with low conditional probability
 260 under the current policy. These tokens correspond to decisions where small parameter changes can
 261 induce large changes in likelihood, and therefore often dominate gradient variance. Weighting by
 262 surprisal concentrates learning signal on such high-sensitivity decisions.

263 **Entropy-Reduction Weighting.** Entropy-reduction weighting emphasizes tokens that sharply
 264 reduce predictive entropy. These tokens correspond to commitment points in generation, where the
 265 model transitions from a diffuse distribution over continuations to a more concentrated one. This
 266 mirrors the role of branching points in tree-based reasoning methods, but is computed locally from
 267 the model’s predictive distribution without explicit tree construction.

268 4.5 Bias–Variance Tradeoff

269 Both advantage estimation and intrinsic token weighting introduce bias in exchange for variance
 270 reduction. Advantage estimation does so by relying on a learned value function, whose bias can be
 271 substantial when the underlying state abstraction is weak. Intrinsic token weighting introduces bias
 272 by reshaping the contribution of token-level gradients based on heuristic salience measures. The
 273 empirical question is therefore not whether bias is introduced, but whether the bias–variance tradeoff
 274 is favorable.

275 This motivates the empirical comparison implemented in the repository: if intrinsic token weighting
 276 improves accuracy, $\text{pass}@k$, or optimization behavior under matched settings, then the bias it
 277 introduces may be more benign than the bias induced by ill-defined value targets over text prefixes.
 278 The point is not that weighting is unbiased, but that its inductive bias may be better aligned with
 279 token-level credit assignment in language generation.

280 4.6 Summary

281 This perspective reframes advantage estimation as one particular approach to credit redistribution in
 282 policy gradient methods. Intrinsic token weighting offers an alternative that leverages the structure
 283 of the language model’s predictive distribution, without assuming the existence of meaningful state
 284 values. In settings where prefixes do not form a stable state space, such value-free mechanisms may
 285 provide a more appropriate inductive bias.

5 Experiments

5.1 Scope and Reproducibility

The repository now implements the experiments described in this paper as runnable Qwen-based training jobs rather than as a synthetic benchmark. We intentionally do not report fabricated numbers here: the code, configs, and evaluation harness are included in the repository, and the actual tables should be populated from completed runs.

5.2 Models

Our reference math model is Qwen/Qwen2.5-Math-1.5B-Instruct. Our reference code model is Qwen/Qwen2.5-Coder-1.5B-Instruct. Both are trained with LoRA adapters and optional 4-bit loading so that the experiments fit on modest hardware.

5.3 Tasks and Rewards

GSM8K. We use the gsm8k/main split as the primary benchmark. Each prompt is formatted as a reasoning task with an explicit final-answer convention. The reward is binary: it is 1 iff the extracted final numeric answer matches the reference answer after normalization, and 0 otherwise.

MBPP. We also implement a code benchmark using MBPP. The model is prompted to return Python code, and reward is binary pass/fail under the task’s unit tests. Because arbitrary code execution is involved, the repository uses a lightweight timeout-based subprocess harness; users may substitute a stronger sandbox when running larger sweeps.

5.4 Methods Compared

The implemented training variants are:

- **RLOO + uniform weighting**
- **RLOO + surprisal weighting**
- **RLOO + entropy-reduction weighting**
- **GRPO + uniform weighting**
- **GRPO + surprisal weighting**
- **GRPO + entropy-reduction weighting**

All methods share the same rollout count, prompt formatting, optimizer family, and decoding hyperparameters within a task. This isolates the effect of token-level credit redistribution from unrelated implementation changes.

5.5 Training Procedure

The main experiment entrypoint is `experiment/run_experiment.py`. For each minibatch of prompts, we sample K completions from the current policy, compute binary rewards after full generation, construct either RLOO or GRPO prompt-level advantages, and then apply the weighted token-level objective from Section ?? . The code recomputes token log-probabilities under teacher forcing for the sampled completions and then applies one of three within-trajectory weighting rules: uniform, surprisal, or entropy reduction.

The repository ships explicit YAML configs under `experiment/configs/experiments/`. For example:

- `qwen25-math-1.5b-gsm8k-rloo-uniform.yaml`
- `qwen25-math-1.5b-gsm8k-rloo-surprisal.yaml`
- `qwen25-math-1.5b-gsm8k-rloo-entropy.yaml`
- `qwen25-math-1.5b-gsm8k-grpo-uniform.yaml`

- qwen25-math-1.5b-gsm8k-grpo-surprisal.yaml
- qwen25-math-1.5b-gsm8k-grpo-entropy.yaml

Equivalent MBPP configs are provided for the code setting.

5.6 Metrics

The code records the metrics needed for the paper’s main comparisons:

- **Greedy accuracy:** deterministic correctness on held-out prompts.
- **Pass@ k :** success rate over k sampled completions.
- **Reward mean / variance:** rollout-level training diagnostics.
- **Completion length:** to monitor drift in response style.
- **Token-weight statistics:** average weight concentration under each intrinsic rule.

For GSM8K, correctness is exact-answer match. For MBPP, correctness is unit-test pass rate.

5.7 How to Populate Final Tables

Because the current revision is meant to be honest about what has and has not been run, the paper tables should be filled only after executing the provided configs. A minimal first pass is to run the six GSM8K configurations above and report greedy accuracy plus pass@4 on the chosen evaluation slice. The same procedure can then be extended to MBPP.

5.8 Compute Considerations

The implementation is intentionally compact: intrinsic token weights are computed from quantities already produced by the policy during rollout or the subsequent teacher-forced forward pass. No learned critic, reward model, or search tree is required. This keeps the experiments aligned with the paper’s central claim while remaining practical to run on limited hardware.

References

- [1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2024.
- [2] Meng Cao, Shuyuan Zhang, Xiao-Wen Chang, and Doina Precup. SCAR: Shapley credit assignment for more efficient RLHF, 2025.
- [3] Yekun Chai, Haoran Sun, Huang Fang, Shuohuan Wang, Yu Sun, and Hua Wu. MA-RLHF: Reinforcement learning from human feedback with macro actions. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [4] Hongzhan Chen, Tao Yang, Shiping Gao, Ruijun Chen, Xiaojun Quan, Hongtao Tian, and Ting Yao. Discriminative policy optimization for token-level reward models, 2025.
- [5] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30*, 2017.
- [6] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiawen Yuan, Hanshen Chen, Kangcheng Zhang, Xiang Lv, Shang Wang, Sinan Yao, Xu Han, Hao Peng, Yejun Cheng, Zhiyuan Liu, Maosong Sun, Bowen Zhou, and Ning Ding. Process reinforcement through implicit rewards, 2025.
- [7] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025.

- [8] Falko Helm, Nico Daheim, and Iryna Gurevych. Token weighting for long-range language modeling. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1440–1459, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics.
- [9] Jian Hu, Jason Klein Liu, and Shen Wei. REINFORCE++: An efficient RLHF algorithm with robustness to both prompt and reward models, 2025.
- [10] Zishun Huang, Dongqi Pu, Xianglin Chen, Jiayi Li, Zehan Qi, Zhihan Zhou, Xiaoxuan Liu, Zhengbang Zhu, Yuying Wang, and Junbo Zhao. Red: Unleashing token-level rewards from holistic feedback via reward redistribution, 2024.
- [11] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. Vineppo: Unlocking rl potential for llm reasoning through refined credit assignment, 2024.
- [12] Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. ReMax: A simple, effective, and efficient reinforcement learning method for aligning large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024.
- [13] Youssef Mroueh, Nicolas Dupuis, Brian Belgodere, Apoorva Nitsure, Mattia Rigotti, Kristjan Greenewald, Jiri Navratil, Jerret Ross, and Jesus Rios. Revisiting group relative policy optimization: Insights into on-policy and off-policy training, 2025.
- [14] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35*, 2022.
- [15] Prasanna Parthasarathi, Mathieu Reymond, Boxing Chen, Yufei Cui, and Sarath Chandar. Grpo- λ : Credit assignment improves llm reasoning, 2025.
- [16] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1889–1897, Lille, France, 07–09 Jul 2015. PMLR.
- [17] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.
- [18] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.
- [19] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.
- [20] Lloyd S. Shapley. A value for n-person games. In Harold W. Kuhn and Albert W. Tucker, editors, *Contributions to the Theory of Games II*, pages 307–317. Princeton University Press, Princeton, 1953.
- [21] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F. Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems 33*, 2020.
- [22] Hieu Tran, Zonghai Yao, and Hong Yu. Exploiting tree structure for credit assignment in RL training of LLMs, 2025.
- [23] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning, 2025.

- [24] Xumeng Wen, Zihan Liu, Shun Zheng, Zhijian Xu, Shengyu Ye, Zhirong Wu, Xiao Liang, Yang Wang, Junjie Li, Ziming Miao, Jiang Bian, and Mao Yang. Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base LLMs, 2025.
- [25] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, may 1992.
- [26] Wei Xiong, Jiarui Yao, Yuhui Xu, Bo Pang, Lei Wang, Doyen Sahoo, Junnan Li, Nan Jiang, Tong Zhang, Caiming Xiong, and Hanze Dong. A minimalist approach to LLM reasoning: from rejection sampling to reinforce, 2025.
- [27] Shentao Yang, Shujian Zhang, Congying Xia, Yihao Feng, Caiming Xiong, and Mingyuan Zhou. Preference-grounded token-level guidance for language model fine-tuning. In Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine, editors, *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023*, 2023.
- [28] Hee Suk Yoon, Eunseop Yoon, Mark A. Hasegawa-Johnson, Sungwoong Kim, and Chang D. Yoo. ConfPO: Exploiting policy model confidence for critical token selection in preference optimization. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 72641–72655. PMLR, 2025.
- [29] Chong Zhang, Yue Deng, Xiang Lin, Bin Wang, Dianwen Ng, Hai Ye, Xingxuan Li, Yao Xiao, Zhanfeng Mo, Qi Zhang, and Lidong Bing. 100 days after deepseek-r1: A survey on replication studies and more directions for reasoning language models, 2025.
- [30] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization, 2025.

A Technical appendices and supplementary material

Technical appendices with additional results, figures, graphs, and proofs may be submitted with the paper submission before the full submission deadline (see above). You can upload a ZIP file for videos or code, but do not upload a separate PDF file for the appendix. There is no page limit for the technical appendices.

Note: Think of the appendix as “optional reading” for reviewers. The paper must be able to stand alone without the appendix; for example, adding critical experiments that support the main claims to an appendix is inappropriate.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.
- Keep the checklist subsection headings, questions/answers and guidelines below.
- Do not modify the questions and only use the provided macros for your answers.

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO] [N/A].

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO] [N/A].

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO] [N/A].

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO] [N/A].

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO] [N/A].

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]** [N/A].

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]** [N/A].

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]** [N/A].

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: **[TODO]** [N/A].

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer **[No]**, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]** [N/A].

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or **[No]**, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: **[TODO]** [N/A].

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]** [N/A].

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]** [N/A].

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]** [N/A].

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO] [N/A].

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO] [N/A].

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.